

Problemas Racso de Static Trees y Strings

Enunciado oficial resumido, solucion de examen, correctitud, complejidad y pseudocodigo

Como leer este PDF

Este documento corrige y expande la guia anterior. Para cada problema se uso la pagina oficial del juez como fuente de verdad y se redactó un **resumen fiel en español**. No se copia literalmente todo el statement oficial; se mantiene el link para consulta. Lo importante para estudiar es que ahora cada problema tiene:

- que pide exactamente;
- como reconocer la estructura;
- solucion de examen completa;
- pseudocodigo corto estilo C++;
- prueba de correctitud;
- complejidad y detalles que suelen quitar puntos.

Regla para el examen. Una respuesta no debe ser solo “uso suffix array”. Debe tener: estructura, algoritmo, argumento de correctitud y complejidad. En los problemas mas dificiles se acepta una explicacion conceptual si la idea central esta bien justificada.

Contents

1	Herramientas comunes	4
1.1	Sparse table para RMQ	4
1.2	Suffix array, rank y LCP	4
1.3	LCA y caminos en arbol	4
2	Static RMQ	5
2.1	Enunciado oficial resumido	5
2.2	Como reconocerlo	5
2.3	Solucion de examen	5
2.4	Pseudocodigo	5
2.5	Correctitud	5
2.6	Complejidad	5
3	Suffix array y LCP array	6
3.1	Enunciado oficial resumido	6
3.2	Que debes explicar en examen	6
3.3	Manber-Myers desde cero	6
3.4	Kasai	6
3.5	Pseudocodigo	6
3.6	Correctitud	7
3.7	Complejidad	7

4	Busqueda de subcadenas	8
4.1	Enunciado oficial resumido	8
4.2	Solucion de examen	8
4.3	Por que el intervalo es contiguo	8
4.4	Pseudocodigo	8
4.5	Correctitud	8
4.6	Complejidad	8
5	Contando subcadenas	9
5.1	Enunciado oficial resumido	9
5.2	Solucion de examen	9
5.3	Pseudocodigo	9
5.4	Correctitud	9
5.5	Complejidad	9
6	Subcadenas diferentes	10
6.1	Enunciado oficial resumido	10
6.2	Solucion de examen	10
6.3	Pseudocodigo	10
6.4	Correctitud	10
6.5	Complejidad	10
7	Subcadena en comun mas larga	11
7.1	Enunciado oficial resumido	11
7.2	Solucion de examen	11
7.3	Detalle del empate lexicografico	11
7.4	Pseudocodigo	11
7.5	Correctitud	11
7.6	Complejidad	11
8	Ordenando subcadenas	12
8.1	Enunciado oficial resumido	12
8.2	Solucion de examen	12
8.3	Pseudocodigo	12
8.4	Correctitud	12
8.5	Complejidad	12
9	Bordes	13
9.1	Enunciado oficial resumido	13
9.2	Correccion importante	13
9.3	Idea con suffix array	13
9.4	Como calcular la suma de LCP de todos los pares	13
9.5	Pseudocodigo	13
9.6	Correctitud	13
9.7	Complejidad	14
10	Refran	15
10.1	Enunciado oficial resumido	15
10.2	Como verlo	15
10.3	Solucion	15
10.4	Pseudocodigo	15
10.5	Correctitud	15
10.6	Complejidad	15

11 Substring periodica	16
11.1 Enunciado oficial resumido	16
11.2 Que significa	16
11.3 Teoria necesaria	16
11.4 Solucion teorica aceptable	16
11.5 Pseudocodigo conceptual	16
11.6 Correctitud	17
11.7 Complejidad	17
12 Imperial Roads	18
12.1 Enunciado oficial resumido	18
12.2 Solucion	18
12.3 Pseudocodigo	18
12.4 Correctitud	18
12.5 Complejidad	18
13 Fools and Roads	19
13.1 Enunciado oficial resumido	19
13.2 Solucion	19
13.3 Pseudocodigo	19
13.4 Correctitud	19
13.5 Complejidad	19
14 Frequency of String	20
14.1 Enunciado oficial resumido	20
14.2 Idea principal	20
14.3 Como obtener ocurrencias	20
14.4 Pseudocodigo	20
14.5 Correctitud	20
14.6 Complejidad	20
15 Ruta de respuesta de examen	21

1 Herramientas comunes

1.1 Sparse table para RMQ

La sparse table guarda respuestas de rangos de longitud potencia de dos:

$$st[k][i] = \min(A[i], \dots, A[i + 2^k - 1]).$$

La transición es:

$$st[k][i] = \min(st[k-1][i], st[k-1][i + 2^{k-1}]).$$

Para consultar un rango de longitud len , se toma:

$$k = \lfloor \log_2 len \rfloor.$$

Esto significa que 2^k es la potencia de dos mas grande que cabe dentro del rango. Se usan dos bloques de longitud 2^k : uno pegado al inicio y otro al final. En minimo pueden solaparse porque $\min(x, x) = x$.

1.2 Suffix array, rank y LCP

Un sufijo de S es una parte que empieza en alguna posicion y llega hasta el final. Si $S = \text{banana}$:

sufijos: banana, anana, nana, ana, na, a.

El suffix array SA guarda las posiciones de los sufijos en orden lexicografico. El arreglo inverso rank cumple:

$$\text{rank}[i] = \text{posicion del sufijo } S[i..] \text{ dentro de SA.}$$

El LCP array guarda cuanto comparten sufijos vecinos:

$$\text{LCP}[i] = \text{lcp}(S[\text{SA}[i-1]..], S[\text{SA}[i]..]).$$

Para dos sufijos arbitrarios i, j , si sus ranks son $a < b$:

$$\text{lcp}(i, j) = \min(\text{LCP}[a+1], \dots, \text{LCP}[b]).$$

Por eso se preprocesa RMQ sobre LCP.

1.3 LCA y caminos en arbol

En un arbol fijo, $\text{lca}(u, v)$ parte el camino $u - v$ en dos caminos hacia arriba. Binary lifting guarda:

$$\text{up}[v][j] = \text{ancestro } 2^j \text{ de } v.$$

Si ademas se guarda el maximo de aristas en cada salto, se pueden responder queries de “max edge on path”.

2 Static RMQ

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/694840/problem/A>

2.1 Enunciado oficial resumido

Se da una secuencia de N enteros y Q consultas. Cada consulta contiene l, r con $0 \leq l < r \leq N$ y pide imprimir:

$$\min(a_l, a_{l+1}, \dots, a_{r-1}).$$

El rango es **semiabierto**: incluye l y excluye r . Restricciones relevantes: $N, Q \leq 5 \cdot 10^5$.

2.2 Como reconocerlo

El arreglo no cambia y hay muchas consultas de minimo. Eso grita sparse table: preprocesamiento $O(N \log N)$ y query $O(1)$.

2.3 Solucion de examen

Construyo $st[k][i]$, el minimo del bloque que empieza en i y tiene longitud 2^k . Para responder $[l, r)$, su longitud es $len = r - l$. Tomo $k = \lfloor \log_2 len \rfloor$. Los bloques:

$$[l, l + 2^k) \quad \text{y} \quad [r - 2^k, r)$$

cubren todo $[l, r)$. La respuesta es:

$$\min(st[k][l], st[k][r - 2^k]).$$

2.4 Pseudocodigo

```
build(A):
  lg[1] = 0
  for i = 2..N: lg[i] = lg[i/2] + 1
  for i = 0..N-1: st[0][i] = A[i]
  for k = 1..lg[N]:
    for i = 0; i + (1<<k) <= N; i++:
      st[k][i] = min(st[k-1][i],
                     st[k-1][i + (1<<(k-1))])

query(l, r): // rango [l, r)
  len = r - l
  k = lg[len]
  return min(st[k][l], st[k][r - (1<<k)])
```

2.5 Correctitud

Por induccion, $st[k][i]$ guarda el minimo de su bloque: para $k = 0$ es un solo elemento; para $k > 0$ el bloque se parte en dos mitades ya calculadas. En una consulta, los dos bloques de longitud 2^k cubren todo el rango semiabierto $[l, r)$. El minimo global esta en al menos uno de ellos. Si se solapan, el minimo no cambia porque la operacion es idempotente. Por tanto la respuesta es correcta.

2.6 Complejidad

Preprocesamiento $O(N \log N)$, memoria $O(N \log N)$, query $O(1)$. Detalle de examen: aqui $len = r - l$, no $r - l + 1$, porque el statement usa rango semiabierto $[l, r)$.

3 Suffix array y LCP array

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/A>

3.1 Enunciado oficial resumido

Dada una cadena s de longitud hasta $5 \cdot 10^5$, imprimir dos lineas: el suffix array indexado desde 0 y el LCP array correspondiente. La cadena contiene letras minúsculas.

3.2 Que debes explicar en examen

Debes definir sufijo, suffix array, rank y LCP. Luego puedes proponer Manber-Myers para construir SA y Kasai para construir LCP.

3.3 Manber-Myers desde cero

Manber-Myers no ordena sufijos completos desde el inicio. Ordena prefijos de longitud:

$$1, 2, 4, 8, \dots$$

El arreglo $rank[i]$ significa: el orden/clase del prefijo de longitud actual que empieza en i . Si ya se ordenan prefijos de longitud len , entonces el prefijo de longitud $2len$ que empieza en i se representa por:

$$(rank[i], rank[i + len]).$$

Ordenar esos pares ordena por longitud doble.

3.4 Kasai

Kasai construye el LCP array despues de tener SA. Usa el rank inverso:

$$rank[i] = \text{posicion del sufijo } i \text{ en SA.}$$

Para cada sufijo i , mira su vecino anterior en SA:

$$j = SA[rank[i] - 1].$$

Calcula $\text{lcp}(S[i..], S[j..])$. La optimizacion es que si el LCP anterior era h , al pasar de i a $i + 1$ el LCP baja como mucho en 1, así que se empieza desde $h - 1$.

3.5 Pseudocodigo

```
build_sa(S):
    sa = [0,1,...,n-1]
    rank[i] = S[i]
    for len = 1; len < n; len *= 2:
        sort sa by (rank[i], rank[i+len] or -1)
        newRank[sa[0]] = 0
        for p = 1..n-1:
            newRank[sa[p]] = newRank[sa[p-1]]
            if pair(sa[p]) != pair(sa[p-1]):
                newRank[sa[p]]++
        rank = newRank
    return sa

kasai(S, sa):
    for p = 0..n-1: rank[sa[p]] = p
    h = 0
    for i = 0..n-1:
        if rank[i] == 0: continue
        j = sa[rank[i]-1]
        while i+h < n && j+h < n && S[i+h] == S[j+h]: h++
        lcp[rank[i]] = h
        if h > 0: h--
```

3.6 Correctitud

Manber-Myers es correcto por induccion: si *rank* ordena prefijos de longitud len , entonces el par de ranks ordena las dos mitades de un prefijo de longitud $2len$. Al duplicar hasta cubrir toda la cadena, los prefijos son los sufijos completos. Kasai es correcto porque calcula el LCP de cada sufijo con su vecino lexicografico anterior, y la reutilizacion de h no omite caracteres: al eliminar el primer caracter de dos sufijos, su prefijo comun disminuye como mucho en uno.

3.7 Complejidad

Manber-Myers cuesta $O(n \log n)$ si cada ronda ordena en $O(n)$ con counting/radix sort; con sort comparativo puede ser $O(n \log^2 n)$. Kasai cuesta $O(n)$.

4 Búsqueda de subcadenas

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/B>

4.1 Enunciado oficial resumido

Se da un texto fijo t y n consultas online. Cada consulta trae una cadena s_i y se debe responder si s_i aparece como subcadena de t . La suma total de longitudes de patrones es hasta $3 \cdot 10^5$. El statement pide hacer flush despues de cada respuesta por el formato online.

4.2 Solucion de examen

Construyo el suffix array de t . Una aparicion de P en t equivale a un sufijo de t que empieza con P . En el suffix array, todos los sufijos que tienen prefijo P forman un intervalo contiguo. Hago dos busquedas binarias para encontrar ese intervalo. Si es no vacio, respondo Yes; si es vacio, No.

4.3 Por que el intervalo es contiguo

En orden lexicografico, todos los strings que comparten un prefijo fijo P quedan juntos. Si hubiera un sufijo sin prefijo P entre dos que si lo tienen, en el primer caracter donde difiere tendria que ser simultaneamente mayor que uno y menor que otro con el mismo prefijo, contradiciendo el orden.

4.4 Pseudocodigo

```
has_prefix(pos, P):
    // true si P es prefijo de t[pos..]
    for i = 0..|P|-1:
        if pos+i == n || t[pos+i] != P[i]: return false
    return true

compare_suffix_with_pattern(pos, P):
    // -1 si sufijo < P, 0 si P es prefijo, 1 si sufijo > P
    for i = 0..|P|-1:
        if pos+i == n: return -1
        if t[pos+i] < P[i]: return -1
        if t[pos+i] > P[i]: return 1
    return 0

exists(P):
    L = first position in SA with suffix >= P
    return L < n && has_prefix(SA[L], P)
```

4.5 Correctitud

Cada aparicion de P empieza en alguna posicion i , y entonces el sufijo $t[i..]$ tiene prefijo P . A la inversa, si un sufijo tiene prefijo P , hay una aparicion en su posicion inicial. La busqueda binaria encuentra el primer sufijo no menor que P ; si ese sufijo tiene prefijo P , existe una aparicion, y si no, ningun sufijo posterior puede iniciar el bloque correcto. Por la contiguidad del bloque, la respuesta es correcta.

4.6 Complejidad

Preprocesamiento $O(|t| \log |t|)$. Cada consulta cuesta $O(|P| \log |t|)$ con comparacion directa. El flush no cambia complejidad, solo es requisito de interaccion.

5 Contando subcadenas

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/C>

5.1 Enunciado oficial resumido

Igual que el problema anterior, se da un texto fijo t y consultas online con patrones s_i . Esta vez se debe imprimir cuantas veces aparece s_i como subcadena de t .

5.2 Solucion de examen

Construyo suffix array. Para cada patron P , encuentro con dos busquedas binarias el intervalo $[L, R)$ de sufijos que empiezan con P . El numero de apariciones es:

$$R - L.$$

5.3 Pseudocodigo

```
count_occurrences(P):  
    L = lower_bound_first_suffix_geq(P)  
    R = lower_bound_first_suffix_strictly_after_prefix(P)  
    return R - L
```

5.4 Correctitud

La correspondencia entre ocurrencias y sufijos con prefijo P es biyectiva: cada ocurrencia define un sufijo y cada sufijo con prefijo P define una ocurrencia. Como esos sufijos forman un bloque contiguo en SA, el tamano del bloque es exactamente el numero de ocurrencias.

5.5 Complejidad

$O(|t| \log |t|)$ para construir. Por consulta, $O(|P| \log |t|)$ en la version simple. Puede optimizarse con LCP, pero para respuesta teorica del curso basta el intervalo de suffix array.

6 Subcadenas diferentes

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/D>

6.1 Enunciado oficial resumido

Dada una cadena s de longitud hasta $5 \cdot 10^5$, contar cuantas subcadenas distintas tiene.

6.2 Solucion de examen

Toda subcadena es prefijo de algun sufijo. Si procesamos sufijos en orden de SA, el sufijo $SA[i]$ tiene $n - SA[i]$ prefijos, pero los primeros $LCP[i]$ ya aparecieron como prefijos del sufijo anterior. Entonces:

$$ans = \sum_i ((n - SA[i]) - LCP[i]) = \frac{n(n+1)}{2} - \sum_i LCP[i].$$

6.3 Pseudocodigo

```
distinct_substrings(s):  
    sa = build_sa(s)  
    lcp = kasai(s, sa)  
    ans = 1LL*n*(n+1)/2  
    for i = 1..n-1:  
        ans -= lcp[i]  
    return ans
```

6.4 Correctitud

Cada subcadena aparece como prefijo de al menos un sufijo. En orden lexicografico, cuando llegamos a un sufijo, los prefijos que ya fueron vistos son exactamente los que comparte con el sufijo anterior, medidos por $LCP[i]$. Los prefijos mas largos son nuevos. Por tanto sumar aportes nuevos cuenta cada subcadena distinta una sola vez.

6.5 Complejidad

Construccion de SA + LCP: $O(n \log n)$ con Manber-Myers y Kasai. Conteo: $O(n)$.

7 Subcadena en comun mas larga

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/E>

7.1 Enunciado oficial resumido

Dadas dos cadenas s, t de longitud hasta 10^5 , imprimir una subcadena comun de longitud maxima. Si hay varias, se debe imprimir la lexicograficamente minima.

7.2 Solucion de examen

Concateno:

$$T = s\#t\$,$$

con separadores que no aparecen en las cadenas. Construyo SA y LCP de T . Recorro sufijos vecinos. Si uno viene de s y otro de t , su LCP es una subcadena comun. Tomo la mayor longitud; en empate, la lexicograficamente menor.

7.3 Detalle del empate lexicografico

Como el suffix array esta en orden lexicografico, las substrings candidatas se pueden comparar por su posicion/rank. Si dos candidatas tienen la misma longitud, se conserva la que aparece antes lexicograficamente. En una implementacion simple se puede comparar las substrings de longitud *best*.

7.4 Pseudocodigo

```
LCS(s, t):
    T = s + "#" + t + "$"
    sa = build_sa(T)
    lcp = kasai(T, sa)
    bestLen = 0; bestPos = -1
    for i = 1..|T|-1:
        if origin(sa[i]) != origin(sa[i-1]):
            candLen = lcp[i]
            candPos = sa[i]
            if candLen > bestLen:
                update
            else if candLen == bestLen:
                choose lexicographically smaller substring
    return T.substr(bestPos, bestLen)
```

7.5 Correctitud

Sea X una subcadena comun optima. Hay un sufijo de s y uno de t que tienen prefijo X . Todos los sufijos con prefijo X forman un bloque en SA. Dentro de ese bloque existe un par adyacente de origen distinto; el LCP de ese par es al menos $|X|$. Por tanto, revisar vecinos de origen distinto encuentra una subcadena comun de longitud maxima. El criterio de desempate elige la menor en orden lexicografico.

7.6 Complejidad

$O((|s| + |t|) \log(|s| + |t|))$ con Manber-Myers. Recorrido lineal.

8 Ordenando subcadenas

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/F>

8.1 Enunciado oficial resumido

Se da una cadena s de longitud hasta $4 \cdot 10^5$ con caracteres ASCII imprimibles. Luego se dan n subcadenas por pares (l, r) , indexados desde 1. Se deben imprimir esos pares ordenados por el valor lexicografico de la subcadena. Si dos subcadenas son iguales como palabra, se desempata por el par: primero menor l , luego menor r .

8.2 Solucion de examen

Para comparar $s[l_1..r_1]$ y $s[l_2..r_2]$, calculo:

$$x = \text{lcp}(l_1, l_2)$$

con SA + LCP + RMQ, recortado por la longitud minima. Si x cubre la mas corta, gana la mas corta; si tienen igual longitud, desempata por (l, r) . Si no, comparo el primer caracter distinto.

8.3 Pseudocodigo

```
lcp_suffix(i, j):
    if i == j: return n-i
    a = rank[i]; b = rank[j]
    if a > b: swap(a,b)
    return rmq_lcp(a+1, b)

less_substring(A=(l1,r1), B=(l2,r2)):
    len1 = r1-l1+1; len2 = r2-l2+1
    x = min(lcp_suffix(l1,l2), min(len1,len2))
    if x < min(len1,len2):
        return s[l1+x] < s[l2+x]
    if len1 != len2: return len1 < len2
    return A < B // desempate por l, luego r
```

8.4 Correctitud

El primer caracter donde difieren dos strings determina el orden lexicografico. El LCP de los sufijos desde l_1 y l_2 da exactamente cuantos caracteres iniciales coinciden. Si la coincidencia termina antes que ambas subcadenas, el siguiente caracter decide. Si una subcadena se acaba, entonces es prefijo de la otra y la mas corta es menor. Si son iguales, el statement pide desempatar por el par.

8.5 Complejidad

Preprocesamiento $O(n \log n)$. Cada comparacion $O(1)$. Ordenar q subcadenas cuesta $O(q \log q)$.

9 Bordes

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/G>

9.1 Enunciado oficial resumido

Una cadena t es borde de s si es a la vez prefijo y sufijo de s . El statement incluye la cadena vacia como borde. Dada una cadena s de longitud hasta $4 \cdot 10^5$, se pide el numero total de bordes de **todas las subcadenas** de s .

9.2 Correccion importante

Este problema no pide solamente los bordes de la cadena completa. Pide sumar, para cada subcadena de s , cuantos bordes tiene. Por eso una solucion simple con KMP sobre la cadena completa no alcanza.

9.3 Idea con suffix array

Cada subcadena tiene siempre el borde vacio. Eso aporta:

$$\frac{n(n+1)}{2}.$$

Ahora contamos bordes no vacios. Un borde no vacio de longitud b en una subcadena aparece cuando hay dos substrings iguales de longitud b : una al inicio y otra al final de esa subcadena. Por cada par de sufijos (i, j) , cada longitud $b \leq \text{lcp}(i, j)$ produce un borde no vacio. Entonces:

$$ans = \frac{n(n+1)}{2} + \sum_{i < j} \text{lcp}(i, j).$$

9.4 Como calcular la suma de LCP de todos los pares

Si dos sufijos tienen ranks $a < b$, su LCP es el minimo en el intervalo $\text{LCP}[a+1..b]$. Por tanto:

$$\sum_{i < j} \text{lcp}(i, j)$$

es la suma de minimos de todos los subarreglos del arreglo LCP. Se calcula con pila monotona en $O(n)$.

9.5 Pseudocodigo

```
sum_pair_lcp(lcp[1..n-1]):
    stack = empty // pares (height, count)
    cur = 0
    total = 0
    for h in lcp[1..n-1]:
        cnt = 1
        while stack not empty && stack.top.height >= h:
            cur -= stack.top.height * stack.top.count
            cnt += stack.top.count
        pop
        push (h, cnt)
        cur += h * cnt
        total += cur
    return total

answer = n*(n+1)/2 + sum_pair_lcp(lcp)
```

9.6 Correctitud

Para cada par de posiciones $i < j$, si los sufijos comparten L caracteres, entonces para cada $b = 1..L$ las substrings de longitud b que empiezan en i y j son iguales. Esas dos copias forman el prefijo y sufijo de la subcadena $s[i..j+b-1]$, asi que aportan un borde no vacio. A la inversa, cualquier borde no vacio de cualquier subcadena define exactamente un par de substrings iguales en sus extremos, y por tanto un par de sufijos con LCP suficiente. La pila monotona suma el minimo de todos los intervalos de LCP, que equivale a la suma de LCP de todos los pares de sufijos.

9.7 Complejidad

Construir SA + LCP: $O(n \log n)$ o $O(n)$ segun algoritmo. Sumar con pila: $O(n)$. Usar `long long`.

10 Refran

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/H>

10.1 Enunciado oficial resumido

Se da un arreglo de n enteros entre 1 y m , con $m \leq 10$. Una subsecuencia consecutiva del arreglo se llama refran. Hay que encontrar un refran que maximice:

$$\text{longitud} \times \text{numero de apariciones.}$$

Se imprime ese valor maximo, la longitud y el refran.

10.2 Como verlo

Aunque el input es un arreglo de enteros, se trata igual que un string sobre un alfabeto pequeno. Las “subcadenas” son subarreglos consecutivos. Se construye suffix array del arreglo.

10.3 Solucion

Una subcadena que aparece f veces corresponde a f sufijos consecutivos que comparten un prefijo de cierta longitud. Si un intervalo de LCP tiene minimo h y cubre w barras, entonces hay $w + 1$ sufijos que comparten un prefijo de longitud h . El valor candidato es:

$$h \cdot (w + 1).$$

Esto es el mismo patron que “largest rectangle in histogram”, usando LCP como histograma.

10.4 Pseudocodigo

```
best_refrain():
    sa = suffix_array_over_integer_array(a)
    lcp = kasai_over_array(a, sa)
    bestValue = n; bestLen = n; bestPos = 0 // frecuencia 1
    use monotonic stack over lcp:
        when height h is popped over width w:
            freq = w + 1
            value = h * freq
            if value > bestValue:
                bestValue = value
                bestLen = h
                bestPos = sa[some suffix in interval]
    print a[bestPos .. bestPos+bestLen-1]
```

10.5 Correctitud

Si un subarreglo X aparece f veces, los sufijos que empiezan en esas apariciones comparten X como prefijo y forman un bloque en el suffix array. La longitud comun maxima del bloque es el minimo LCP interno. La pila monotona considera cada posible minimo de un bloque maximal, asi que considera el bloque que representa cualquier refran optimo. Por tanto encuentra el maximo producto.

10.6 Complejidad

$O(n \log n)$ con construccion simple de suffix array, $O(n)$ para LCP y pila. El alfabeto pequeno ayuda, pero la idea teorica es suffix array + LCP histogram.

11 Substring periodica

Link oficial. <https://codeforces.com/group/AmklRC8lhZ/contest/695758/problem/I>

11.1 Enunciado oficial resumido

Dada una cadena s de longitud hasta $4 \cdot 10^5$, encontrar el mayor entero k tal que exista una subcadena no vacia que pueda escribirse como la concatenacion de k cadenas iguales.

11.2 Que significa

Buscamos el maximo exponente de una repeticion. Por ejemplo, si aparece

$$d^k = d + d + \dots + d,$$

entonces esa subcadena tiene periodo $|d|$ y longitud $k|d|$.

11.3 Teoria necesaria

Para probar si una zona tiene periodo p , necesitamos comparar substrings desplazadas:

$$s[l..r-p] = s[l+p..r].$$

Con SA + LCP + RMQ, esta igualdad se verifica en $O(1)$. Para encontrar la respuesta maxima de forma eficiente, una solucion completa enumera **runs** o repeticiones maximales. Un run es un intervalo maximal con un periodo minimo p y longitud al menos $2p$.

11.4 Solucion teorica aceptable

Preprocesa LCP hacia la derecha y LCP en la cadena invertida para obtener LCS (longest common suffix). Para un posible periodo p y una frontera entre dos bloques iguales, calculas:

$$R = \text{lcp}(i, i+p)$$

extension hacia la derecha, y:

$$L = \text{LCS}(i-1, i+p-1)$$

extension hacia la izquierda. Entonces el bloque maximal con periodo p alrededor de esa frontera tiene longitud aproximadamente:

$$L + p + R.$$

El numero de repeticiones candidatas es:

$$\left\lfloor \frac{L + p + R}{p} \right\rfloor.$$

Una implementacion completa debe enumerar fronteras representativas sin probar todos los pares; esto se hace con algoritmos de runs o con tecnicas de divide-and-conquer. La idea de examen es justificar la verificacion de periodos con LCP/LCS.

11.5 Pseudocodigo conceptual

```
is_period(l, len, p):
    return lcp_suffix(l, l+p) >= len-p

max_power_by_runs():
    build LCP structure on S and reversed(S)
    ans = 1
    for each maximal run (L, R, p):
        ans = max(ans, (R-L+1) / p)
    return ans
```


11.6 Correctitud

Una subcadena es d^k si tiene periodo $p = |d|$ y longitud al menos kp . La igualdad desplazada por p caracteriza exactamente a las cadenas con periodo p . Los runs son intervalos maximales donde esa propiedad se mantiene; cualquier repeticion esta contenida en algun run de su periodo. Por eso maximizar $\lfloor \text{longitud} / \text{runPeriod} \rfloor$ sobre runs da el mayor k .

11.7 Complejidad

La verificacion de igualdad por LCP es $O(1)$ despues de preprocesar. Enumerar runs puede hacerse en $O(n)$ o $O(n \log n)$ segun algoritmo. Este es un problema dificil; para examen, asegura explicar periodo, LCP/LCS y runs antes que intentar escribir implementacion completa.

12 Imperial Roads

Link oficial. <https://codeforces.com/gym/101889/problem/I>

12.1 Enunciado oficial resumido

Se tiene un grafo no dirigido, conectado y ponderado con N ciudades y R carreteras. Para cada consulta se da una carretera específica que debe ser reparada/incluida. Se pide el costo mínimo de un conjunto de carreteras que mantenga conectadas todas las ciudades e incluya esa carretera. Es decir: MST forzando una arista.

12.2 Solucion

Construyo un MST T con peso total W . Para una arista consultada $e = (u, v, w)$, si la agrego a T , se crea un unico ciclo formado por e mas el camino entre u y v en T . Para volver a tener un arbol que incluya e , debo quitar una arista de ese ciclo. Para minimizar el costo, quito la arista de mayor peso en el camino $u - v$ del MST.

$$ans = W + w - \text{maxEdge}(u, v).$$

12.3 Pseudocodigo

```
build:
  T, W = Kruskal(graph)
  root T
  preprocess up[v][j] and mx[v][j]

maxEdge(u, v):
  ans = 0
  lift deeper node, updating ans
  lift both until LCA, updating ans
  include last edges to LCA
  return ans

query(edge u,v,w):
  return W + w - maxEdge(u,v)
```

12.4 Correctitud

Agregar e al MST crea exactamente un ciclo. Cualquier arbol generador que incluya e y use las aristas de T salvo una debe eliminar una arista del camino $u - v$. El costo nuevo es $W + w - x$, donde x es la arista eliminada. Para minimizar, se maximiza x , por eso se quita la mayor arista del camino. Binary lifting augmentado calcula ese maximo correctamente porque descompone el camino en saltos de potencias de dos.

12.5 Complejidad

Kruskal $O(R \log R)$. Preprocesamiento LCA $O(N \log N)$. Cada consulta $O(\log N)$.

13 Fools and Roads

Link oficial. <https://codeforces.com/problemset/problem/191/C>

13.1 Enunciado oficial resumido

Se da un árbol de n ciudades y $n - 1$ carreteras. Luego se conocen varias rutas simples usadas por personas, cada una entre dos ciudades. Para cada carretera se debe contar cuantas rutas pasan por ella.

13.2 Solucion

Uso diferencia en árbol. Para una ruta (u, v) :

$$cnt[u] + +, \quad cnt[v] + +, \quad cnt[lca(u, v)] - = 2.$$

Luego hago DFS postorden. El acumulado que sube desde un hijo x al padre es la cantidad de rutas que usan la arista $(parent[x], x)$.

13.3 Pseudocodigo

```
for each route (u,v):
    a = lca(u,v)
    cnt[u]++
    cnt[v]++
    cnt[a] -= 2

dfs(u,p):
    sum = cnt[u]
    for v in adj[u], v != p:
        child = dfs(v,u)
        ans[edge(u,v)] = child
        sum += child
    return sum
```

13.4 Correctitud

Los $+1$ en los extremos suben durante la acumulacion. Restar 2 en el LCA cancela ambas contribuciones antes de que pasen por encima del LCA. Por eso se marcan exactamente las aristas del camino $u - v$. Como la suma es lineal, todas las rutas pueden procesarse juntas.

13.5 Complejidad

Preprocesar LCA $O(n \log n)$. Cada ruta $O(\log n)$. DFS final $O(n)$.

14 Frequency of String

Link oficial. <https://codeforces.com/problemset/problem/963/D>

14.1 Enunciado oficial resumido

Se da un string s y n consultas. Cada consulta contiene un entero k_i y un patron m_i . Se pide la longitud minima de una substring de s que contenga al menos k_i ocurrencias de m_i . Si m_i aparece menos de k_i veces, se imprime -1 . Los patrones son distintos y la suma de sus longitudes es hasta 10^5 .

14.2 Idea principal

Si conocemos las posiciones de ocurrencia de m :

$$p_1 < p_2 < \dots < p_t,$$

entonces la menor ventana que contiene k ocurrencias consecutivas $p_i, \dots, p_{i+k-1}$ mide:

$$p_{i+k-1} - p_i + |m|.$$

La respuesta es el minimo de esa formula.

14.3 Como obtener ocurrencias

Como hay muchos patrones y texto fijo, Aho-Corasick es natural:

1. Insertar todos los patrones en un trie.
2. Construir failure links.
3. Recorrer s una vez.
4. Registrar en que posiciones termina cada patron.

Tambien puede razonarse con suffix array, pero Aho-Corasick se ajusta mejor a muchos patrones distintos.

14.4 Pseudocodigo

```
answer(pattern id, k):
    occ = occurrences[id] // posiciones de inicio ordenadas
    if occ.size() < k: return -1
    ans = INF
    len = pattern_length[id]
    for i = 0; i+k-1 < occ.size(); i++:
        ans = min(ans, occ[i+k-1] - occ[i] + len)
    return ans
```

14.5 Correctitud

Cualquier substring que contiene k ocurrencias tiene una primera y una k -esima ocurrencia entre las posiciones ordenadas. Si esas son p_i y p_{i+k-1} , la substring debe cubrir desde p_i hasta el final de la ocurrencia en p_{i+k-1} , longitud al menos $p_{i+k-1} - p_i + |m|$. Esa ventana existe y contiene esas k ocurrencias, por lo que revisar ocurrencias consecutivas da el optimo.

14.6 Complejidad

Aho-Corasick: $O(|s| + \sum |m_i| + occ)$ segun implementacion de salida. Las ventanas se calculan linealmente en el numero de ocurrencias de cada patron. Detalle: las ocurrencias pueden solaparse y deben contarse.

15 Ruta de respuesta de examen

Para cada problema de este PDF, tu respuesta debe sonar así:

1. Identifico el objeto fijo: arreglo, string o árbol.
2. Elijo la estructura: sparse table, suffix array/LCP, LCA, Aho-Corasick.
3. Explico que guarda la estructura.
4. Transformo la query al lenguaje de la estructura.
5. Pruebo por que esa transformacion es exacta.
6. Cierro complejidad de preproceso, query y memoria.

Prioridad real

Si falta tiempo, domina primero:

1. Static RMQ.
2. Suffix array, rank, LCP y Kasai.
3. Búsqueda/conteo de patrones por intervalo de suffix array.
4. Subcadenas distintas y longest common substring.
5. Comparador de substrings con LCP+RMQ.
6. Imperial Roads y Fools and Roads para LCA aplicado.
7. Borders/Refran/Periodic/Frequency como problemas avanzados.